

# Konzeptabgabe

**Themenbereich:** LLM Performance Vergleich (Feature Realisation)

**Werkzeug:** CASCADE

**Gruppenmitglieder:** Bruno Vincent Hemoura, Alexander Hristov, Caspar Moritz Klein

**Gruppe:** 01

## 1 Forschungsfrage

Wie kann CASCADE so erweitert werden, dass LLM-Nutzung, Laufzeitverhalten und Resultate der Pipeline strukturiert erfasst werden, um verschiedene Modelle, Prompts und Ausführungsschritte vergleichbar zu machen?

## 2 Abstract

CASCADE generiert und validiert Tests mithilfe einer mehrstufigen Pipeline aus Extraktion, Analyse, Generierung und Ausführung. Für einen fundierten LLM-Performance-Vergleich fehlte bisher eine zentrale Erfassung von Tokenverbrauch, Laufzeiten und Fehlern. Im Rahmen dieser Feature-Realisierung wird ein leichtgewichtiger Metrics-Mechanismus für CASCADE implementiert. Er soll Events wie bestimmte Utility-Aufrufe oder Fehler protokollieren und am Ende eines Runs eine Zusammenfassung generieren. Besonders liegt der Fokus auf LLM-Aufrufen sowie zentralen Pipeline-Schritten. Das bisherige Logging von Fehlern soll dabei ausgebaut werden, sodass Schwachstellen und Zeitverluste einfacher zu erkennen sind.

Mit der erfolgten Implementierung werden wir den Mechanismus anhand verschiedener Modelle demonstrieren.

## 3 Motivation

LLM-basierte Test- und Codegenerierung ist stark abhängig von Modell, Prompting, Eingabegröße und Ausführungsumgebung. Ohne einheitliche Metriken ist schwer nachvollziehbar, ob ein Modell tatsächlich bessere Ergebnisse liefert oder nur höhere Kosten bzw. längere Laufzeiten verursacht. Ziel ist daher eine reproduzierbare Messgrundlage, mit der Tokenverbrauch, Dauer, Fehler und zentrale Teilschritte der CASCADE-Pipeline objektiv verglichen werden können.

Bisher ist schwer zu erkennen, wann Fehler passieren und welche Ursachen diese haben. Es gibt keine einheitliche Konvention, wie Fehler ausgegeben werden, wodurch vor allem in den Komponenten zur Generation, Fehler schwer einzuordnen sind. Deshalb ist die Erweiterung des Loggings auch sinnvoll, weil CASCADE dadurch nicht nur Endergebnisse, sondern auch deren Entstehung nachvollziehbar macht. So können Unterschiede zwischen

Modellen oder Konfigurationen später anhand konkreter Messwerte statt nur anhand beobachteter Resultate bewertet werden.

## 4 Lösungsansatz

Vorgesehen ist ein leichtgewichtiges Metrics-Konzept, das pro CASCADE-Run strukturierte Ereignisse erfasst und anschliessend aggregierte Kennzahlen bereitstellt. Dabei sollen sowohl LLM-spezifische Daten wie Input-, Output- und Total-Tokens als auch allgemeine Performance-Daten wie Laufzeit, Fehler und Pipeline-Phase berücksichtigt werden.

Die Erfassung soll möglichst nah an den relevanten Pipeline-Schritten ansetzen, ohne die bestehende CASCADE-Architektur stark zu verändern. Sinnvoll ist eine Kombination aus expliziten Events für fachliche Informationen und Zeitmessungen für grössere Ausführungsblöcke. Kontextinformationen wie Phase, Methode oder Sample sollen automatisch mitgeführt werden, damit die Messdaten später einem konkreten Pipeline-Schritt zugeordnet werden können.

Die aktuelle Architektur ist grob in Abbildung 1 dargestellt. Problem dabei ist, dass der Pipeline-context, also Informationen über Ein/ Ausgabepfad und Informationen über die einzelnen Komponenten nur innerhalb der Pipeline selbst vorhanden sind, bzw. über Funktionsparameter teilweise weitergegeben werden. Will man nun weitere Metriken hinzufügen, müssen die zu erhebenden Daten entweder direkt über den Ausgabepfad in eine Datei geschrieben werden oder über den Abhängigkeitsbaum nach oben propagiert werden. Dies ist wenig erweiterbar und wird schnell unübersichtlich.

Es sollen deshalb Pipeline-weite Hilfsfunktionen eingeführt werden, welche von allen Komponenten gleichermaßen genutzt werden können um Events wie Fehler, Info-Nachrichten, LLM-Nutzungen oder Zeitverbräuche einheitlich in die gemeinsame Messung einzutragen, welche am Ende des Runs zusammengefasst wird.

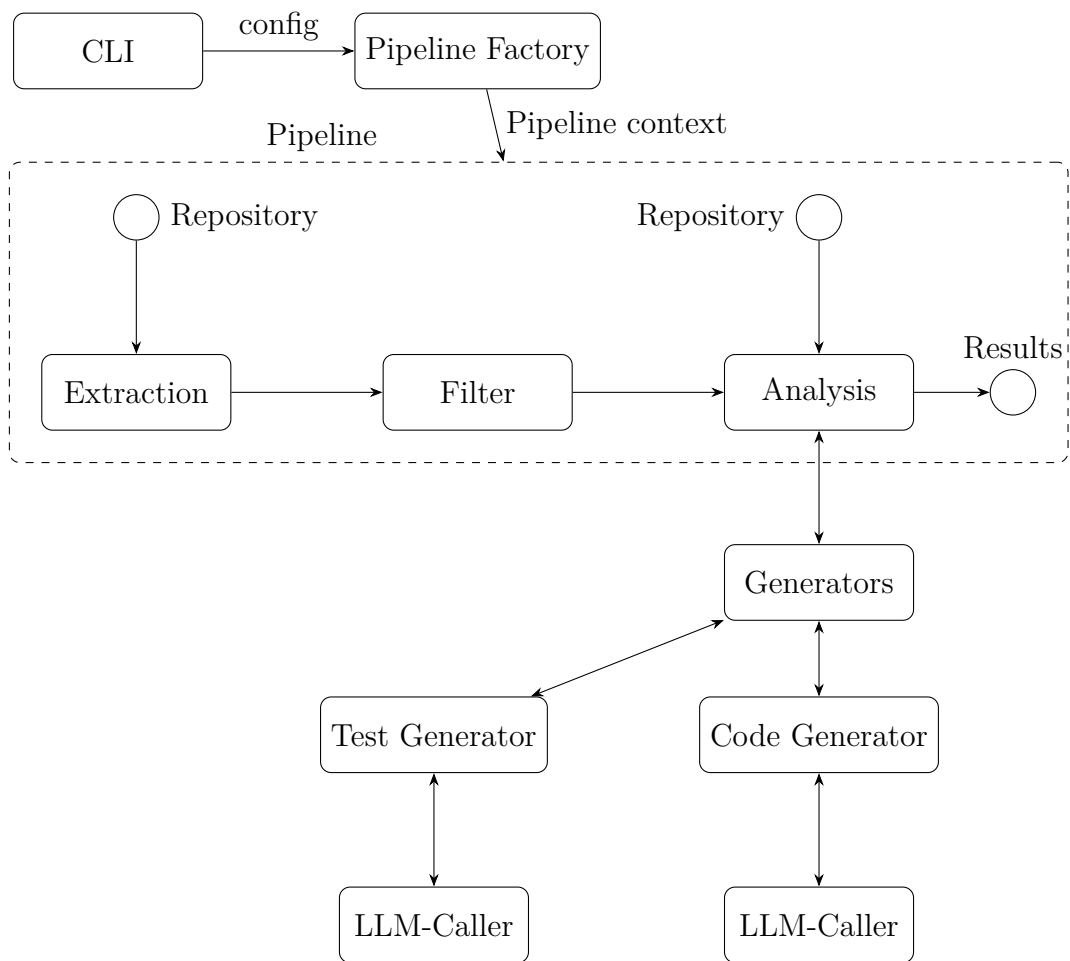


Abbildung 1: Skizze der aktuellen Pipeline-Architektur